difftree

Differentiable decision trees in JAX with built-in uncertainty

John Kitchin
Carnegie Mellon University · Chemical Engineering

Decision trees: the classic recipe

- Recursively split data on **one feature** at a time
- Choose each split **greedily** (best impurity drop now)
- Predict with a constant in each leaf
- Fast, interpretable, handles mixed features
- CART (Breiman et al., 1984), ID3 (Quinlan, 1986)

A hard, axis-aligned split

```
if x[3] < 0.42: go left else: go right
```

Every input follows exactly **one** root-to-leaf path.

Three things classic trees cannot do

Optimize jointly

Greedy growth fixes early splits; later splits cannot fix them. No gradient flows through `if/else`.

Split obliquely

Axis-aligned cuts need staircases to follow a diagonal or curved boundary.

Say how sure they are

A leaf constant gives a point estimate, not a calibrated predictive distribution.

difftree makes every split a smooth function, so the whole tree trains end to end by gradient descent, and keeps that smoothness at test time to report uncertainty.

Where difftree sits

Greedy trees
CART, ID3

›

Oblique trees
OC1, non-greedy

›

Soft / neural trees
DNDF, soft DT, ANT

›

End-to-end trees
anneal $\gamma \to \infty$

›

difftree
finite γ + UQ

- Hehn et al. (2020) train trees end to end, then **anneal to hard splits**
- difftree keeps splits **finitely soft**: routing ambiguity becomes uncertainty
- Leaves are **Gaussians**, so the output is a calibrated mixture

How it works

A soft, oblique split

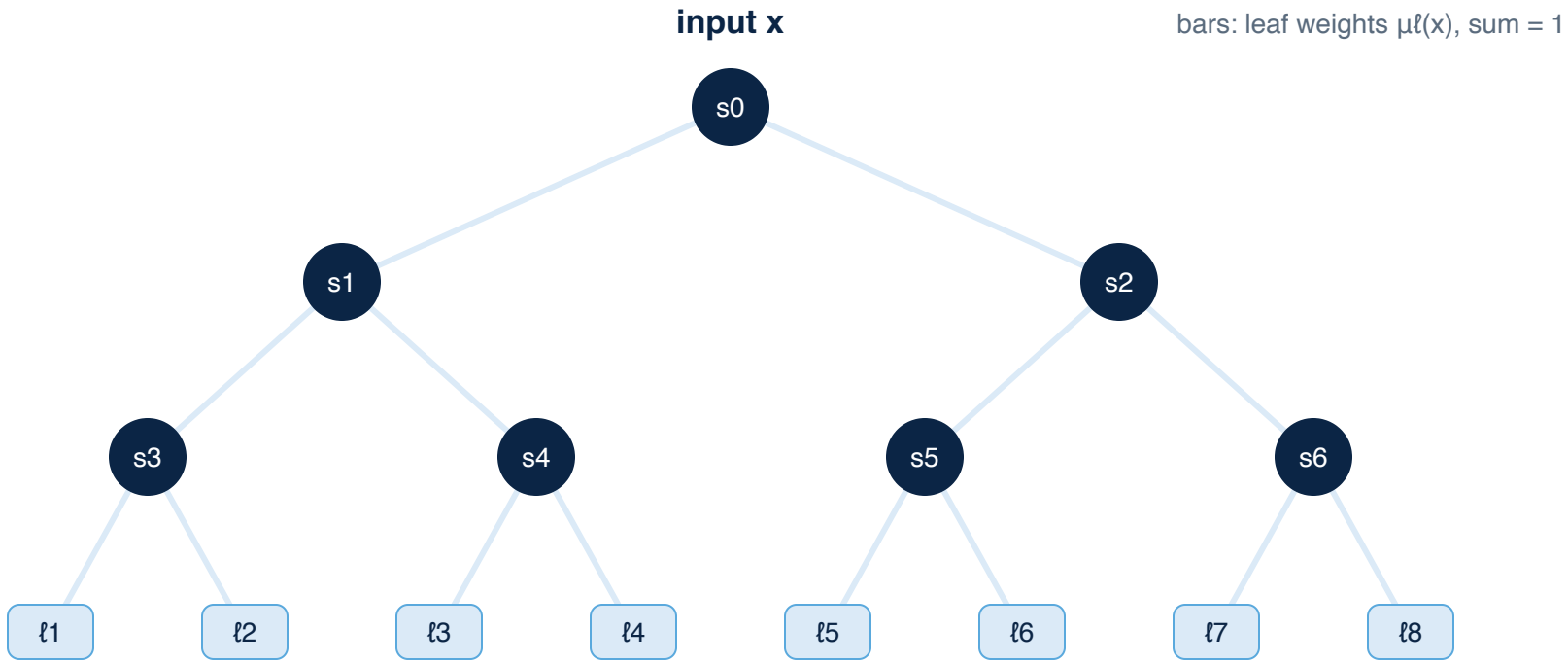
$$s_i(x) = \sigma\big(\gamma_d \, (w_i^\top x + b_i)\big)$$

- w_i: an **oblique** direction (all features at once)
- γ_d: per-depth **steepness**
- s_i = probability of going right
- $\gamma \to \infty$ recovers a hard split

$$\mu_\ell(x) = \prod_{i \in \mathrm{path}(\ell)} s_i^{\,r_i} \, (1 - s_i)^{1 - r_i}$$

Path probabilities over all 2^D leaves sum to one.
Computed in log space for stability.

Soft routing: one input, every leaf



Edge width ∝ probability of reaching that node. Labels are the split probabilities.

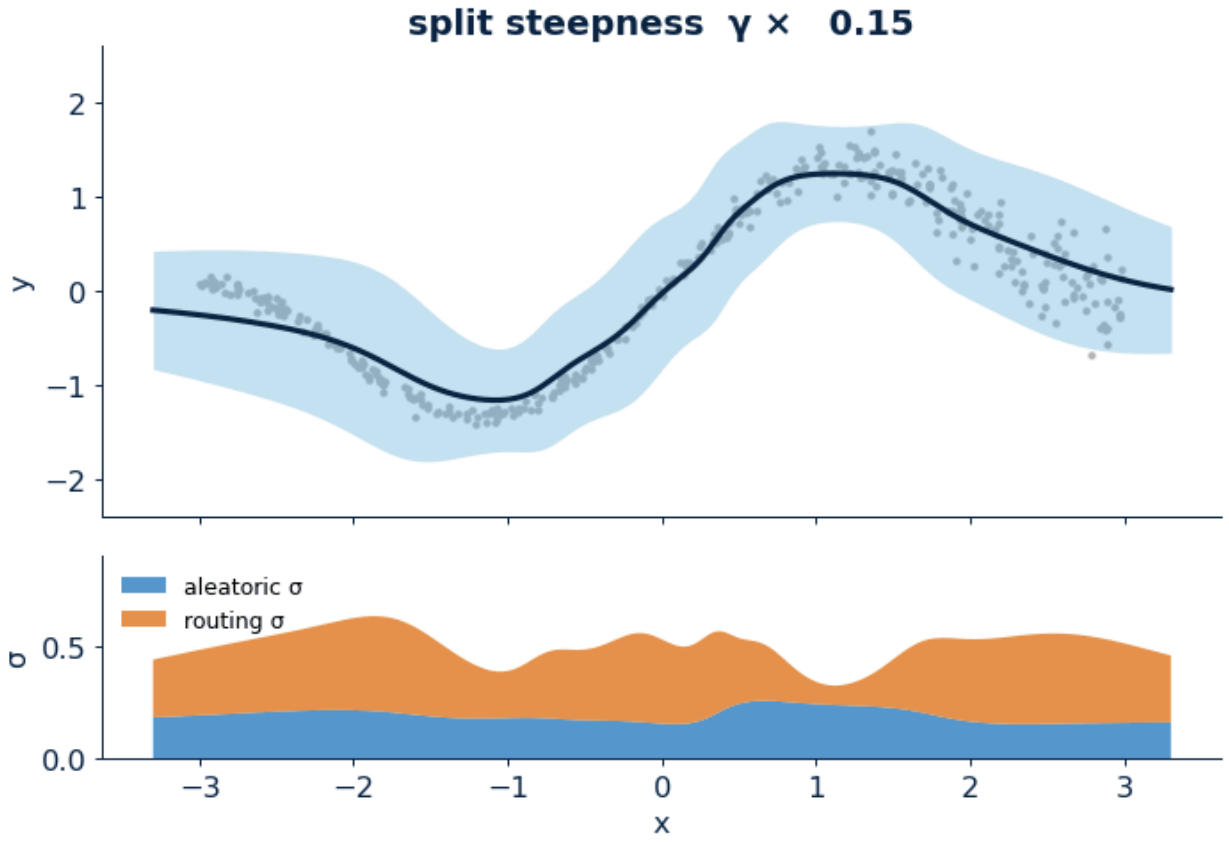
Leaves are Gaussians, so predictions are mixtures

$$p(y \mid x) = \sum_{\ell=1}^{2^D} \mu_\ell(x)\, \mathcal{N}\big(y;\, m_\ell,\, \alpha\, L_\ell L_\ell^\top\big)$$

$$\mathrm{Cov}[y \mid x] = \underbrace{\mathbb{E}_\ell\big[\Sigma_\ell\big]}_{\text{aleatoric (within leaf)}} + \underbrace{\mathrm{Cov}_\ell\big[m_\ell\big]}_{\text{routing (between leaves)}}$$

- Each leaf has its own mean and **Cholesky** covariance (PD by construction)
- Multi-output y gets a full covariance, not just per-output error bars
- Classification: leaves hold logits; entropy splits the same way

Steepness γ controls smoothness and routing uncertainty



- Small γ: smooth mean, large **routing** σ (orange)
- Large γ: piecewise constant, routing σ only at the boundaries
- Calibration picks a **finite** γ per depth
- Aleatoric σ (blue) is the learned noise level

Training: maximize likelihood, then calibrate

Warmup
splits + means, σ frozen

›

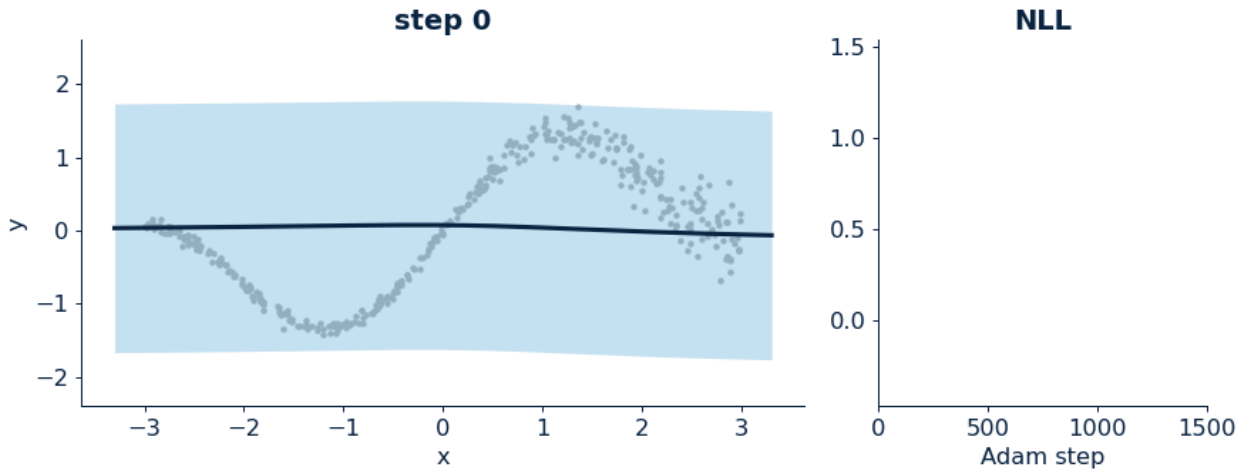
Full NLL
Adam, grad clipping

›

Anneal γ
$1 \rightarrow \gamma_{max}$

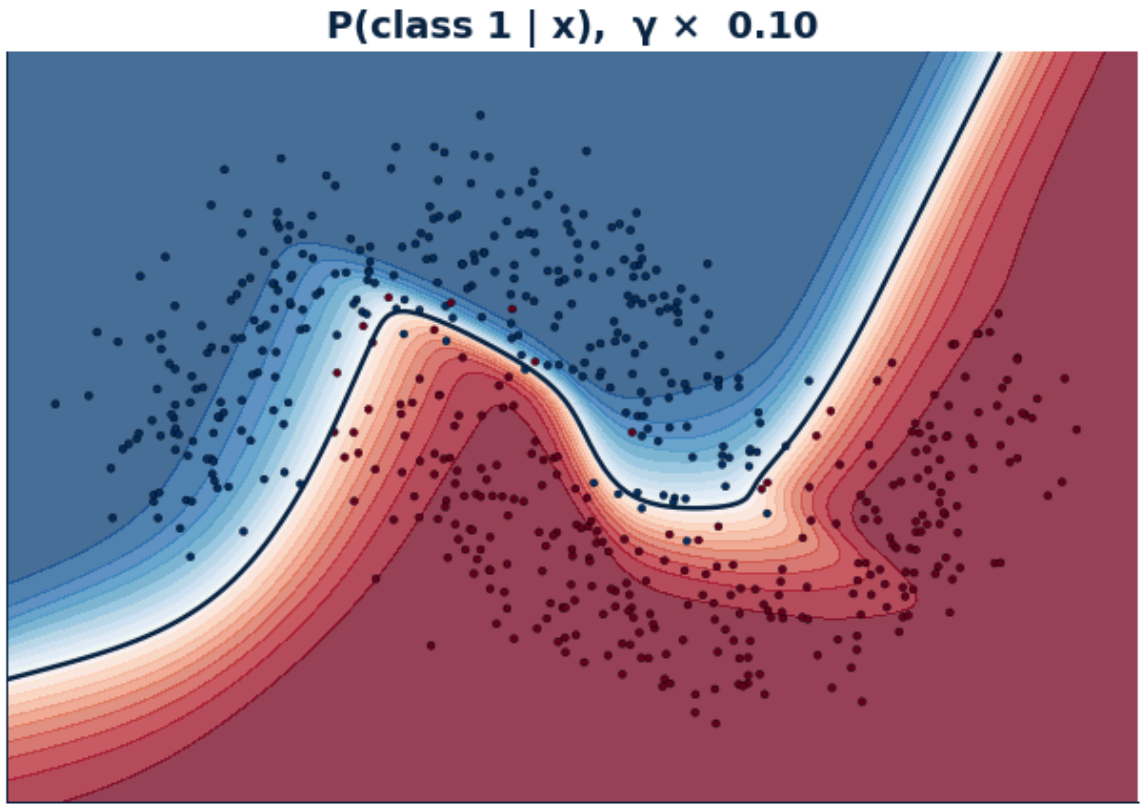
›

Calibrate
fit log γ_d, log α on held-out data



- Loss: $-\log p(y \mid x)$, end to end
- Warmup avoids variance collapse
- Calibration is tree-style **temperature scaling**

Oblique splits bend to the data



- Depth-4 classifier on two moons
- Each cut is a line in **any direction**
- 15 splits trace a curved boundary
- Soft γ: graded probabilities near the boundary
- Hard γ: a polygon partition

Examples

A scikit-learn estimator

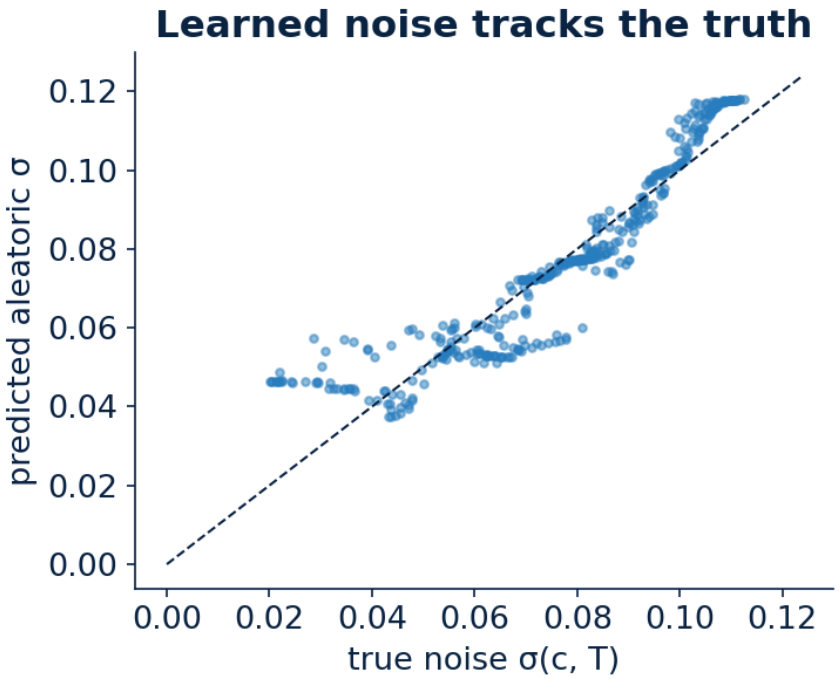
```python
from difftree import DifferentiableTreeRegressor

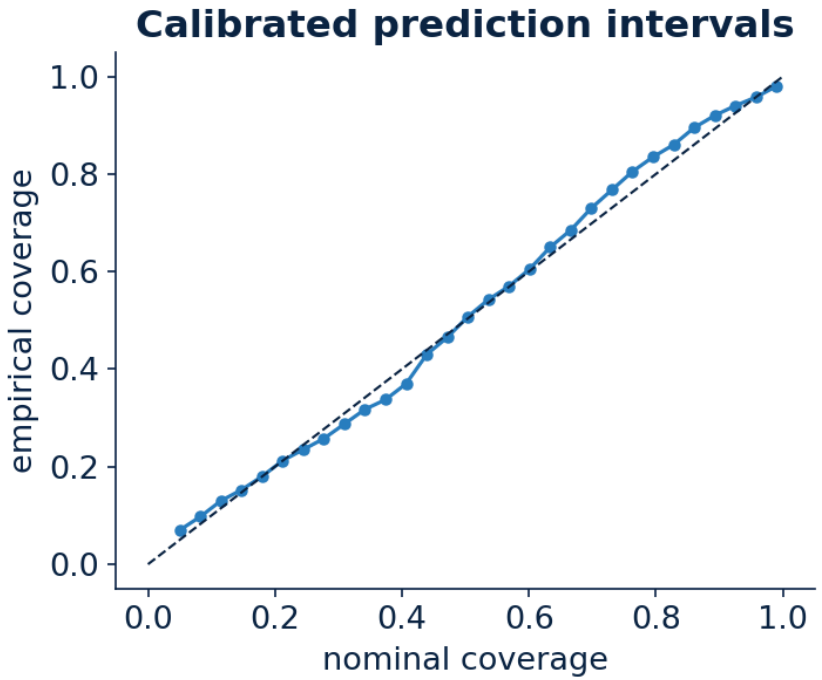
model = DifferentiableTreeRegressor(depth=5, n_epochs=150, random_state=0)
model.fit(X_train, y_train)

mean, std = model.predict(X_test, return_std=True)
mean, std, std_aleatoric, std_routing = model.predict(X_test, return_components=True)
q05, q50, q95 = model.predict_quantile(X_test, q=[0.05, 0.5, 0.95])
samples = model.sample_y(X_test, n_samples=200)
```

Also: `DifferentiableTreeClassifier`, `predict_full_cov`, `predict_conditional`, `inverse_design`, `predict(hard=True)`, `harden`, and the `difftree.export` module. Works with pipelines, CV and grid search.

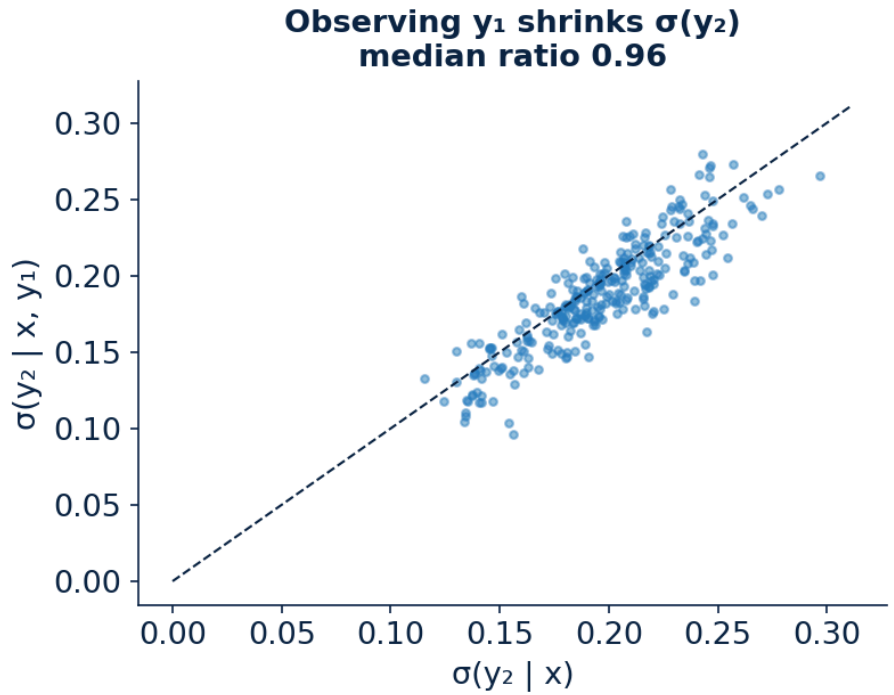
Example 1: a saturating sensor with heteroscedastic noise





Michaelis-Menten response in (c, T) with Arrhenius $K_m(T)$; noise $\sigma = 0.02 + 0.1\, f(c, T)$. Depth 5, 1600 training points. The aleatoric σ recovers the noise structure and intervals are calibrated.

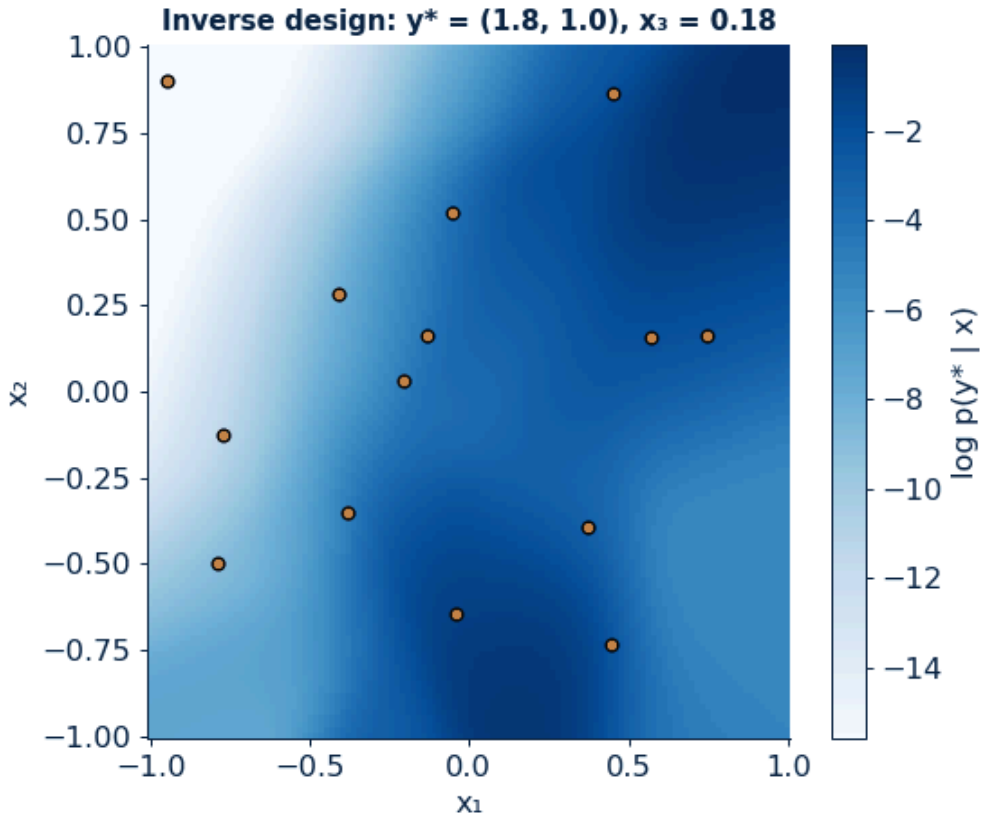
Example 2: knowing one output tightens the other



- Two correlated properties y_1, y_2 from three inputs
- $p(y_2 \mid x, y_1)$ in **closed form**: Schur complement per leaf, leaf weights reweighted by evidence
- Conditional σ is smaller for most inputs, but modestly here (median ratio 0.96)

```
m, C, logev = model.predict_conditional(
    X, observed={0: y1})
```

Example 3: inverse design by gradient ascent



- Target properties $y^* = (1.8,\ 1.0)$
- Maximize $\log p(y^* \mid x)$ over x
- Tree is smooth in x, so use `jax.grad`
- Multi-start Adam finds **several** solutions
- Partial targets and box bounds supported

Piecewise-affine surrogates for optimization

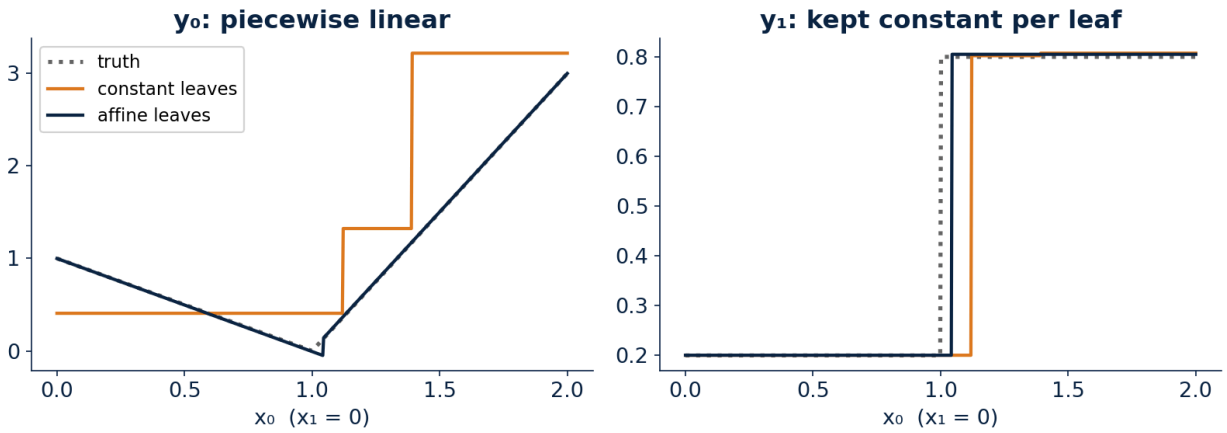
From a soft tree to a MILP surrogate

Affine leaves $y = A_\ell x + b_\ell$ › **Harden** $\gamma \to \infty$, refit leaves › **Export** polytopes + JSON spec › **Embed** MILP via jax-ldt › **Check** jumps, repair

> With crisp routing, a tree is a **piecewise-affine** function on polytopes, so it embeds **exactly** in a mixed-integer linear program, with one binary per leaf.

Running example (`notebooks/piecewise_linear_milp.ipynb`): two outputs over $x_0 \in [0, 2]$, $x_1 \in [-50, 50]$, with an oblique kink at $x_0 + 0.02\, x_1 = 1$. Output y_0 is piecewise linear and y_1 is piecewise constant. Depth 2, 60 training points.

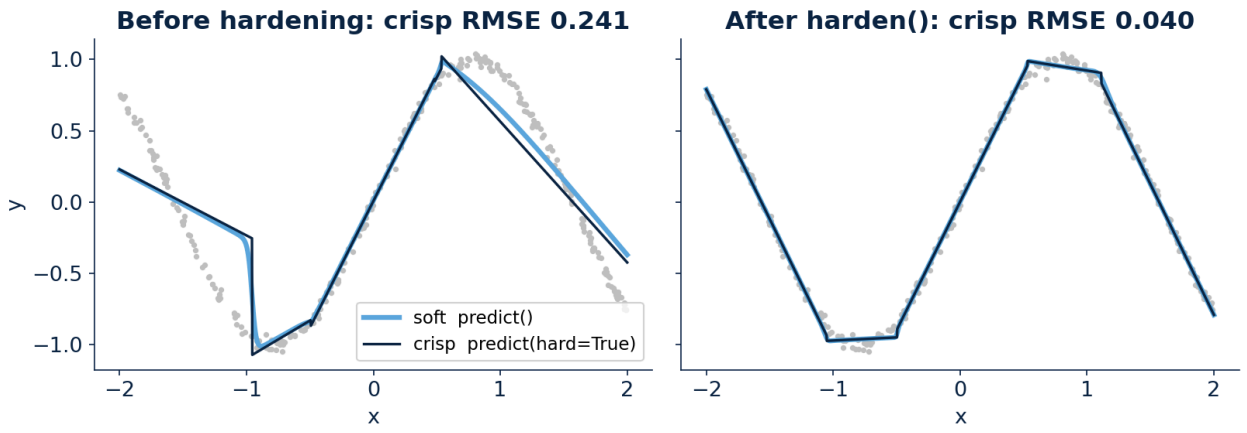
Affine leaves: one partition, a linear model per leaf



- Leaf mean $m_\ell(x) = \mu_\ell + A_\ell x$; Gaussian noise unchanged, so UQ still works
- Constant leaves need a **staircase** for y_0: crisp RMSE 0.76 vs **0.02**
- `constant_outputs=[1]` keeps y_1 constant per leaf, so blending terms stay linear
- `fit(X, y, dydx=J)` adds Jacobian matching (validate it: no gain here)

```
DifferentiableTreeRegressor(depth=2, leaf_type="affine", constant_outputs=[1], hardening=True)
```

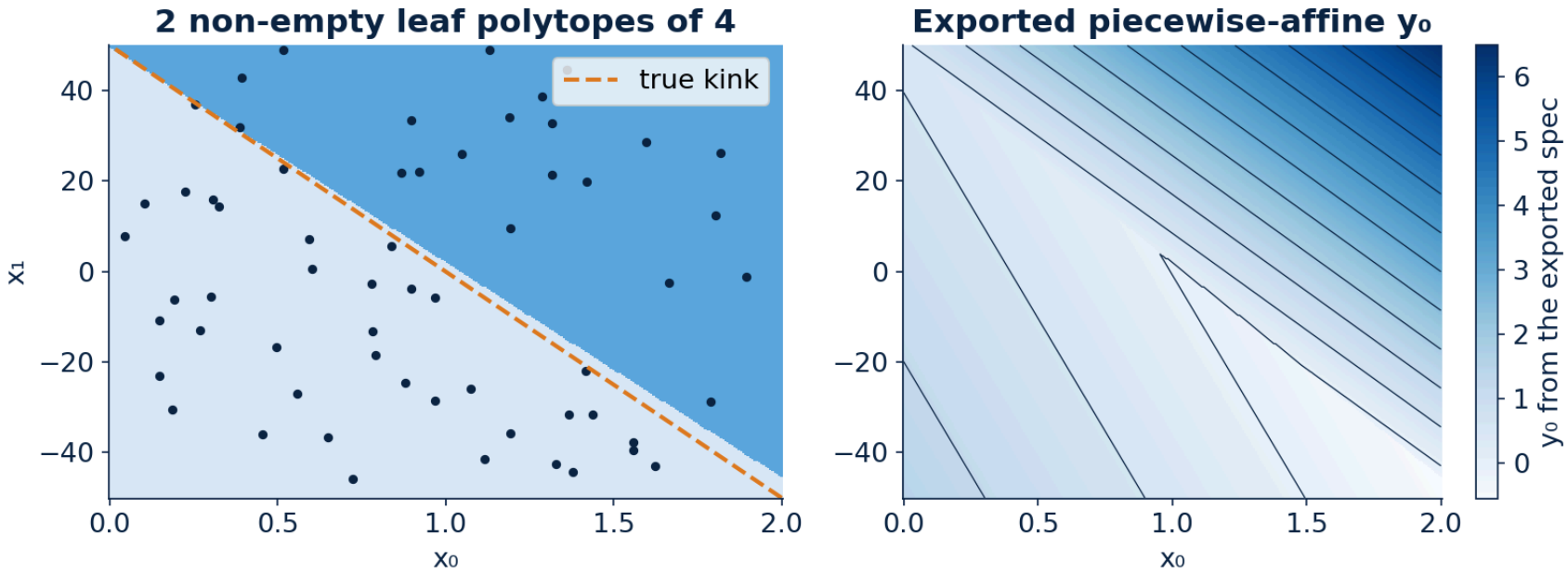
Hardening: make the crisp model the one you trained



- `predict(hard=True)` routes by the **sign** of each split
- A soft fit that blends leaves can be a poor crisp model
- `harden()`: anneal γ, freeze the partition, refit leaves by least squares, recalibrate α
- `soft_hard_gap()` and `hardening_report_` measure the difference

1-D example, $\sin 2x$, depth 3, affine leaves: crisp RMSE 0.24 before hardening, 0.04 after.

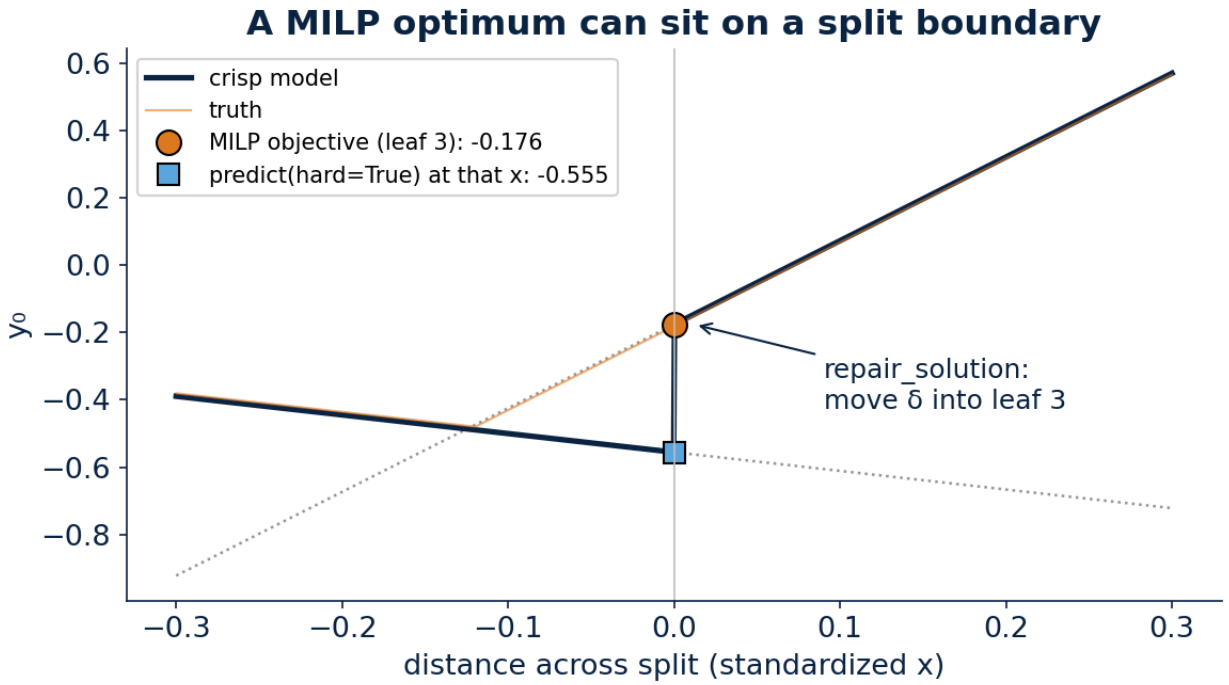
Export: leaf polytopes and a solver-neutral spec



`export.leaf_polytopes`: each leaf as $\{x : G_\ell x \leq h_\ell\}$ with its affine model, in **original units**. One LP per leaf drops empty leaves (here 2 of 4 survive).

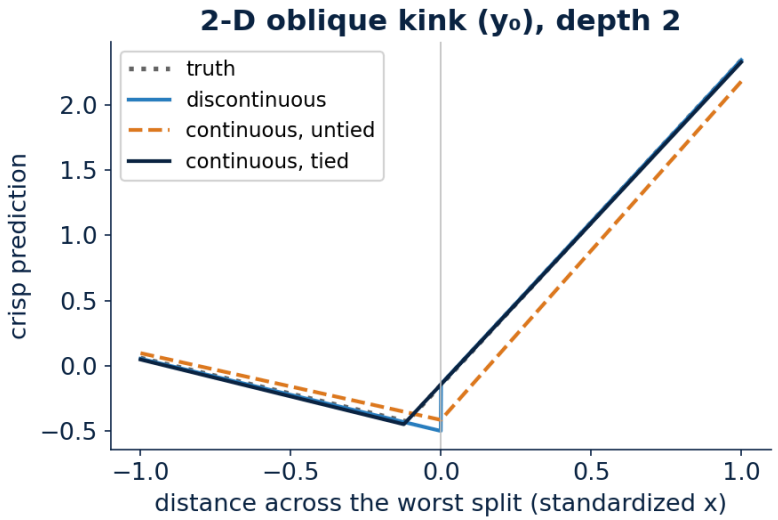
`export.to_json`: jax-ldt v1 spec, read by its discopt adapter. Max $|\text{spec} - \texttt{predict(hard=True)}| = 1.8 \times 10^{-15}$.

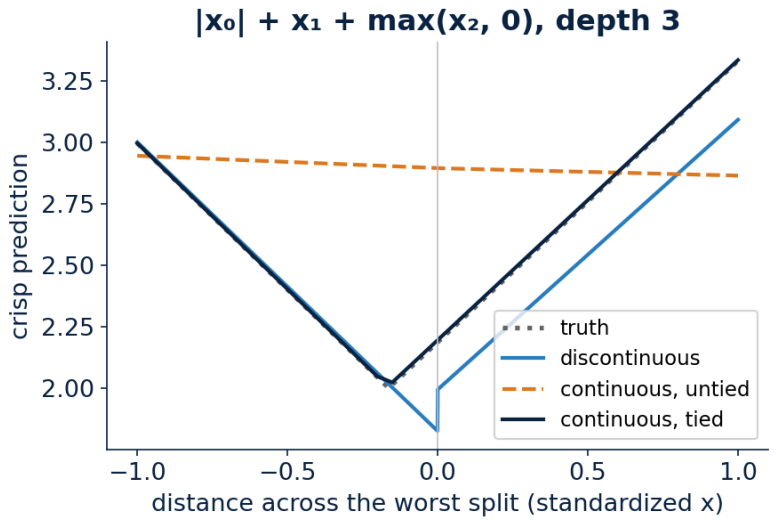
Split boundaries: where the MILP and the tree disagree



- MILP polytopes are **closed**: an optimum on a boundary may use either leaf
- `predict(hard=True)` sends that exact point left, giving a different value
- `export.leaf_from_binaries` reads the leaf the solver chose
- `export.repair_solution` moves x by δ into it (one LP), so both agree
- `export.boundary_jumps` reports every jump; an optimizer will seek out a large one

Removing the jumps: tied splits





`continuous=True` makes neighbouring leaves agree on each facet, which over-constrains an oblique tree (orange). With `tied_splits=True` each depth shares one hyperplane, and the fit is a hinging-hyperplanes model (Breiman, 1993):
$\theta_0^\top [1, x] + \sum_d c_d \max(0,\ w_d^\top x + b_d)$.

Crisp RMSE (max jump ~10^{-15} when continuous)	2-D kink	3-D
discontinuous	0.023	0.049
continuous, untied	0.073	0.583
continuous, tied	**0.007**	**0.004**

Pros, cons and limitations

Strengths

Uncertainty is built in

Mixture predictive with aleatoric / routing split, quantiles, samples, full covariance.

Calibrated

Post-hoc γ and α fit on held-out data; coverage tracks nominal.

Differentiable everywhere

End-to-end training, gradients in *x* for inverse design, composable with JAX.

Multi-output for free

Closed-form conditioning on observed outputs; correlated predictions.

Oblique splits

Fewer leaves for diagonal or curved structure than axis-aligned trees.

Familiar API

scikit-learn estimators; small codebase (JAX + optax).

Weaknesses and limitations

Fixed, full depth

$2^D - 1$ splits; practical depth $\lesssim$ 10 to 12. No grown or pruned trees yet.

Every leaf is evaluated

Soft routing costs $O(2^D)$ per input, not $O(D)$ like a hard tree.

Less interpretable

Oblique, soft splits are harder to read than "$x_3 < 0.42$".

Non-convex training

Seed sensitive: a 1D test gave RMSE 0.11 to 0.34 at depth 4 across 8 seeds.

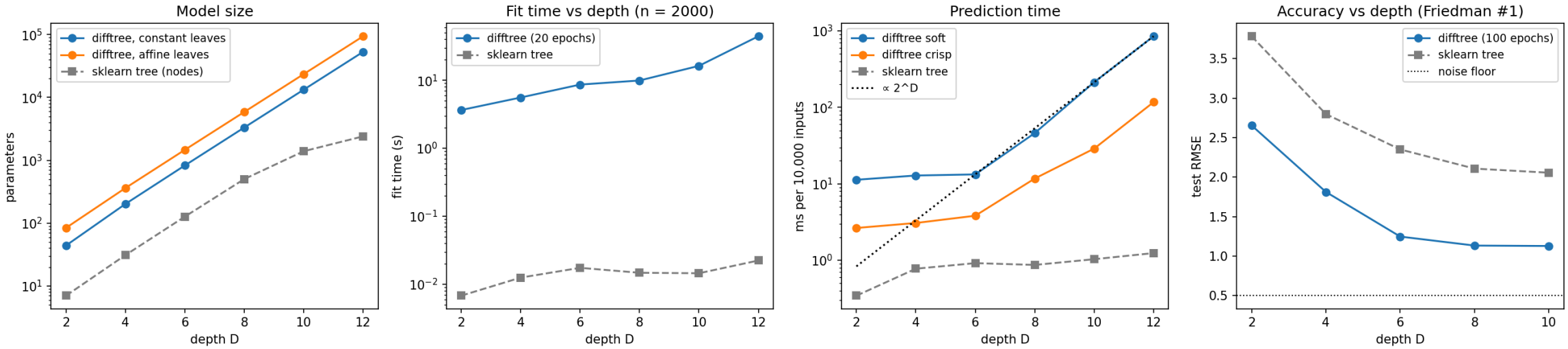
Local inverse design

Multimodal likelihood; relies on multi-start ascent.

Single tree, aleatoric-heavy UQ

No ensemble or Bayesian treatment of parameters; linear splits only for now.

Weaknesses, measured: cost of full depth

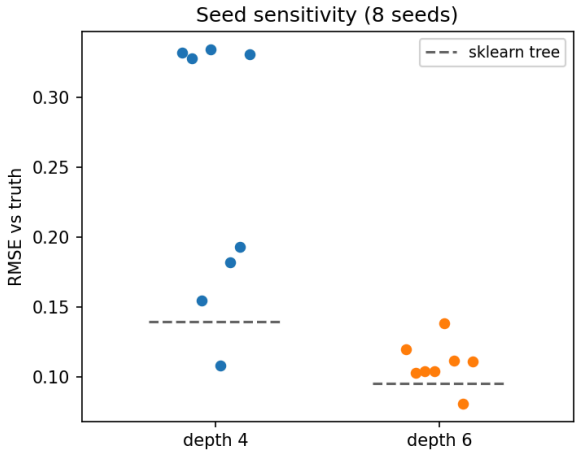


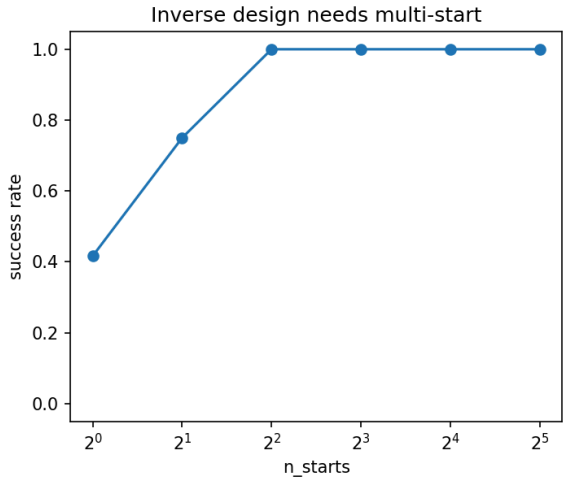
Friedman #1, 10 inputs, CPU. Parameters grow 4x per two levels (53k to 94k at depth 12). Fit time grows about 4x from depth 8 to 12; 64x more samples cost only about 2x.

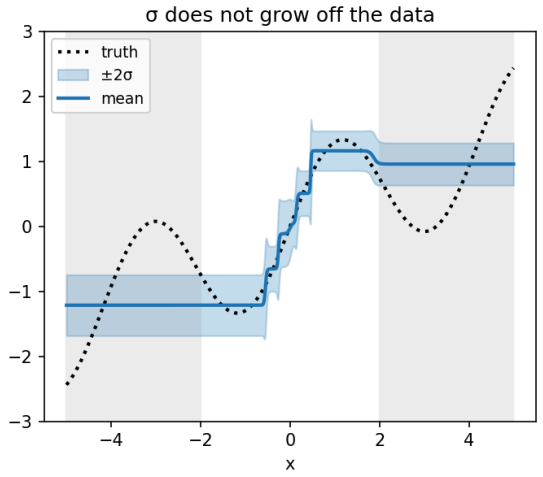
Soft prediction grows as 2^D past depth 6; crisp is 4 to 8x cheaper. Accuracy levels off at depth 8, well above the noise floor.

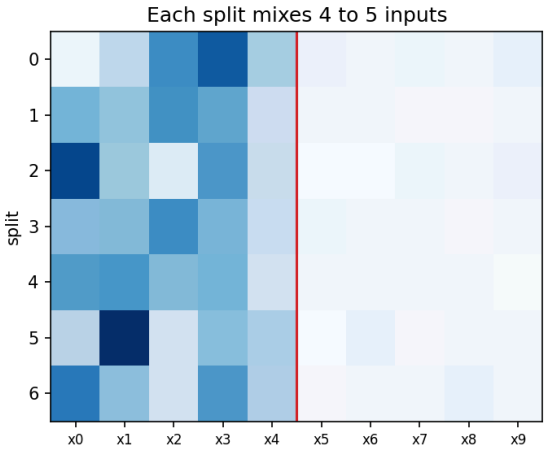
notebooks/scaling.ipynb

Weaknesses, measured: behaviour









Seeds: depth 4 spans RMSE 0.11 to 0.34; picking the seed by held-out R^2 chose the best one. **Inverse design:** 1 start succeeds 42% of the time, 4+ starts always; 32 starts find 6 distinct optima.

Off the data: σ stays at 0.20 while error grows from 0.12 to 0.79 (17% coverage). **Splits:** each mixes 4 to 5 inputs, with little weight on noise inputs. notebooks/failure_modes.ipynb

Take-home

> **difftree** = soft oblique splits + Gaussian leaves + calibration.
> One differentiable model gives a predictive **distribution**, not just a number.

- Good fit: tabular, moderate-size problems that need **calibrated UQ**
- Multi-output properties: **conditioning** and **inverse design** fall out
- Optimization: affine leaves + hardening give a **piecewise-affine** surrogate that exports to a MILP
- Watch: depth limits, seeds, and multi-start for design

Questions?

References

L. Breiman, J. Friedman, R. Olshen, C. Stone, *Classification and Regression Trees*, Chapman & Hall/CRC (1984).

J. R. Quinlan, Induction of decision trees, *Machine Learning* 1, 81 (1986). doi:10.1007/BF00116251

P. Kontschieder et al., Deep Neural Decision Forests, *ICCV* (2015). doi:10.1109/ICCV.2015.172

N. Frosst, G. Hinton, Distilling a Neural Network Into a Soft Decision Tree, arXiv:1711.09784 (2017).

C. Guo et al., On Calibration of Modern Neural Networks, *ICML* (2017). arXiv:1706.04599

T. M. Hehn, J. F. P. Kooij, F. A. Hamprecht, End-to-End Learning of Decision Trees and Forests, *IJCV* 128, 997 (2020). doi:10.1007/s11263-019-01237-6

L. Breiman, Hinging Hyperplanes for Regression, Classification, and Function Approximation, *IEEE Trans. Inf. Theory* 39, 999 (1993). doi:10.1109/18.256506

Figures: `presentation/make_figures.py`, `presentation/make_tree_svg.py`; MILP example from `notebooks/piecewise_linear_milp.ipynb`; measured weaknesses from `notebooks/scaling.ipynb` and `notebooks/failure_modes.ipynb`